Solution: Binary Tree Vertical Order Traversal

Problem Statement

Given the `root` of the binary tree, return the 2D list containing the `vertical order traversal` of the binary tree.

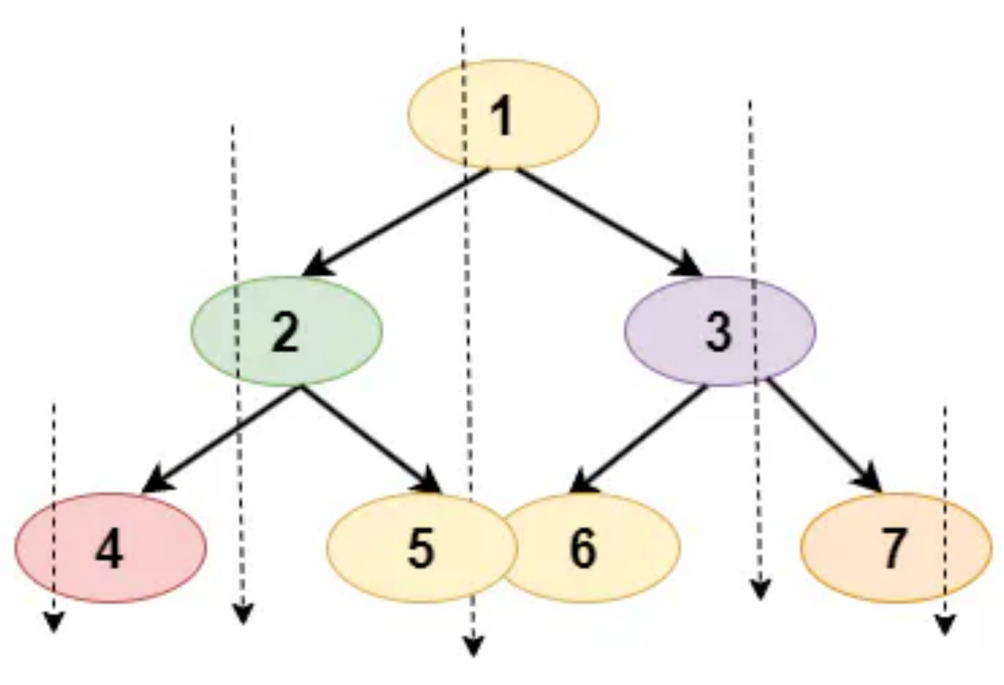
A vertical order traversal of the tree is defined as a `top to bottom, column by column` traversal.

Note: If two nodes are in the same row and column, keep its order from left to right.

Examples

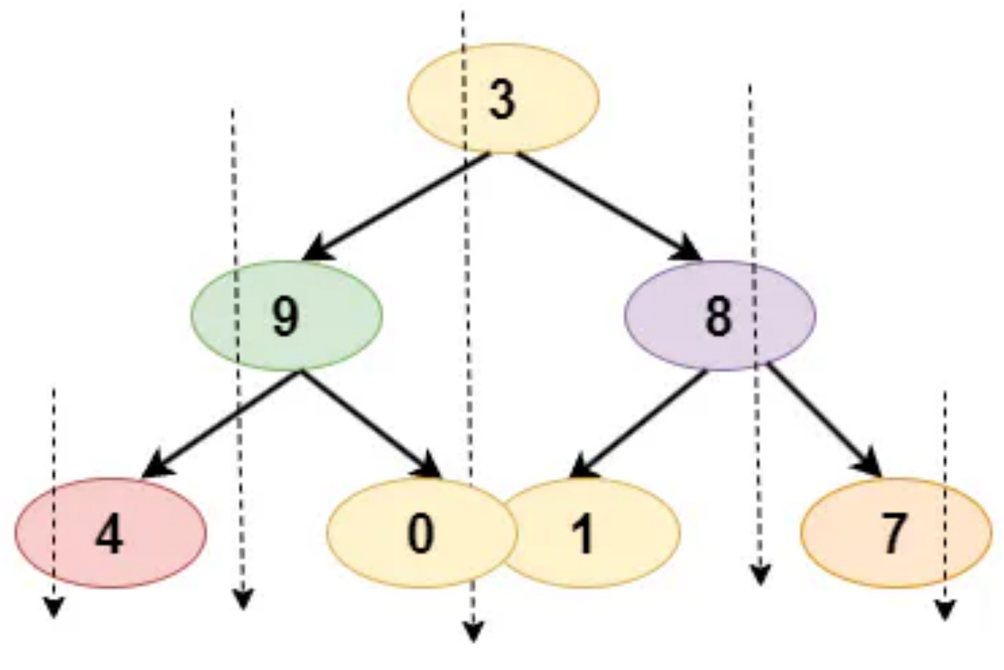
Example 1:

- **Input:** A binary tree: `[1,2,3,4,5,6,7]`
- **Expected Output:** `[[4], [2], [1,5,6], [3], [7]]`
- **Justification:** Nodes 4, 2, 1 with 5 and 6, 3, and 7 are in separate vertical lines. The nodes in each vertical line are listed in the order they appear from top to bottom.



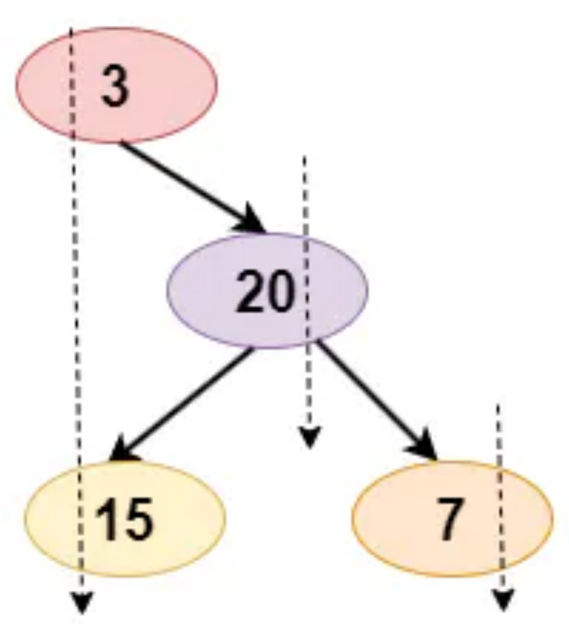
Example 2:

- **Input:** A binary tree: `[3,9,8,4,0,1,7]`
- **Expected Output:** `[[4], [9], [3,0,1], [8], [7]]`
- **Justification:** Nodes are grouped based on their vertical positions. Lower nodes in the same vertical line follow the higher ones.



Example 3:

- **Input:** A binary tree: `[3,null,20,15,7]`
- **Expected Output:** `[[3, 15], [20], [7]]`
- **Justification:** The tree is skewed to the right. The output reflects the vertical traversal from left to right.



Solution

To solve this problem, we'll use a breadth-first search (BFS) strategy combined with a tracking mechanism for the horizontal positions (or 'columns') of the nodes. BFS is ideal because it naturally explores the tree level by level, ensuring that we process nodes on the same level in the correct order.

We'll use a queue to facilitate the BFS, and a hashmap (or dictionary) to keep track of the nodes' column indices. The key point is to associate each node with its respective column index, allowing us to group nodes that are vertically aligned. We'll also keep track of the minimum and maximum column indices encountered, which will guide us in forming the final output list in the correct left-to-right vertical order.

Step-by-step Algorithm

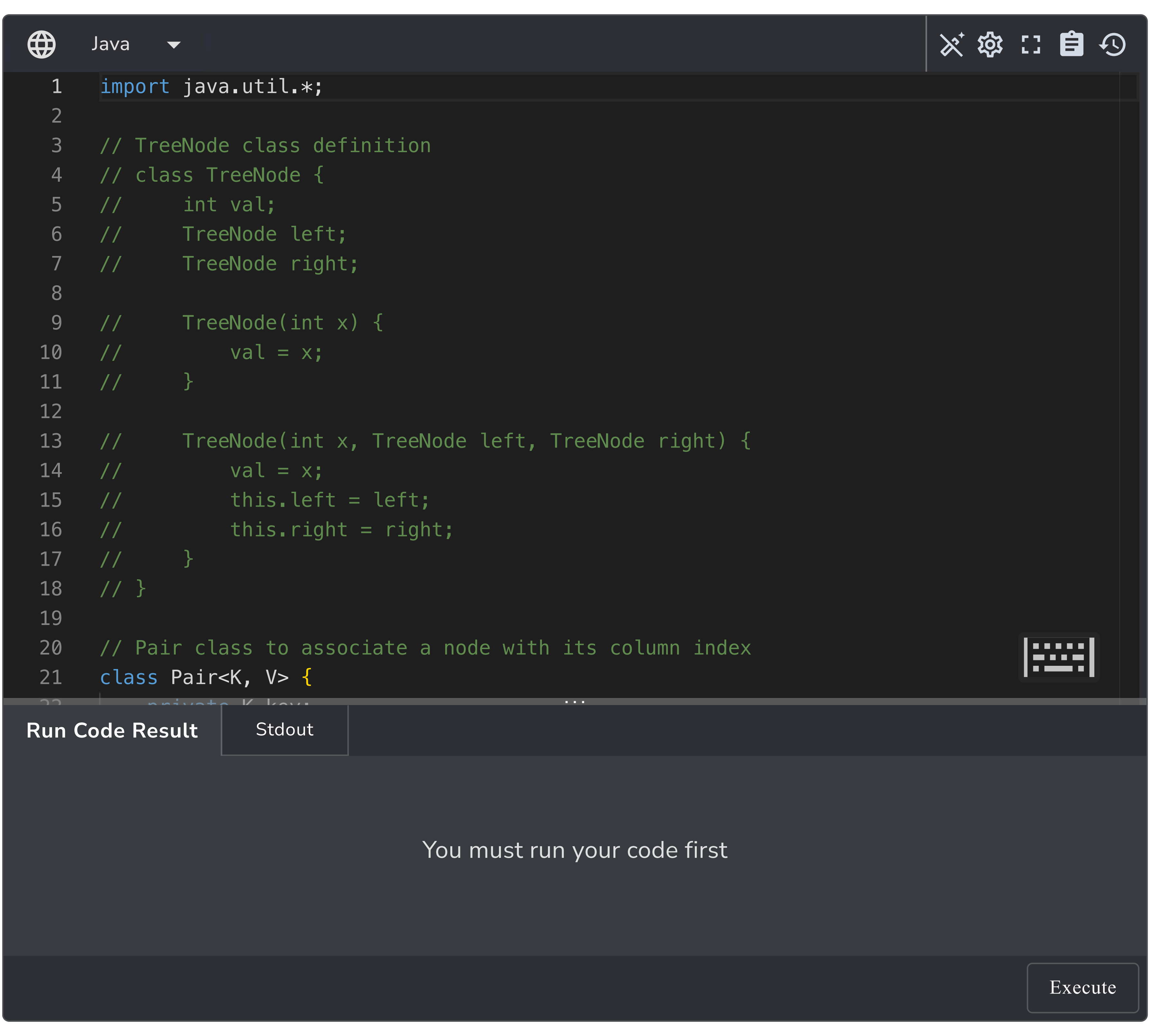
- Initialize a queue to perform BFS and a hashmap to store node values grouped by their column index.
- Start with the root node in the queue, with a column index of 0.
- Perform `BFS`:
  - Dequeue a node from the queue and record its value in the hashmap under its column index.
  - If the node has a `left child`, enqueue it with a column index one less than the current node.
  - If the node has a `right child`, enqueue it with a column index one more than the current node.
  - Update the `minimum` and `maximum column indices` when necessary.
- After completing BFS, iterate from the minimum to the maximum column index, and append the corresponding list of node values from the hashmap to the output list.

Algorithm Walkthrough

Consider the input `[1,2,3,4,5,6,7]`. Let's walk through the algorithm:

- Start with root node `1` at column index `0`.
- Enqueue left child `2` with column index `-1`, and right child `3` with column index `+1`.
- Continue BFS:
  - For node `2`, enqueue its children `4` and `5` with column indices `-2` and `0`, respectively.
  - For node `3`, enqueue its children `6` and `7` with column indices `0` and `+2`, respectively.
- The final column index hashmap looks like: `{-2: [4], -1: [2], 0: [1, 5, 6], 1: [3], 2: [7]}`.
- Iterating over the column indices from `-2` to `2`, we get the final output: `[[4], [2], [1,5,6], [3], [7]]`.

Code



```java
import java.util.*;

// TreeNode class definition
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;

//     TreeNode(int x) {
//         val = x;
//     }

//     TreeNode(int x, TreeNode left, TreeNode right) {
//         val = x;
//         this.left = left;
//         this.right = right;
//     }
// }

// Pair class to associate a node with its column index
class Pair<K, V> {
```

You must run your code first

Complexity Analysis

- **Time Complexity:** The algorithm's time complexity is `O(N)`, where N is the number of nodes in the tree. This is because each node is processed exactly once during the BFS.
- **Space Complexity:** The space complexity is also `O(N)`, as we store all nodes in the hashmap and the queue at

← Previous
Binary Tree Vertical Order Traversal (Medium)

Next →
Buildings With An Ocean View (Medium)

☐ Mark as Completed